

云短信平台优惠活动

题目描述

某云短信厂商，为庆祝国庆，推出充值 **优惠活动** 。

现在给出客户预算，和优惠售价序列，求最多可获得的短信总条数。

输入描述

第一行客户预算 M

- $0 \leq M \leq 10^6$

第二行给出售价表 P1, P2, ... Pn，其中 Pi 为充值 i 元获得的短信条数。

- $1 \leq n \leq 100$
- $1 \leq P_i \leq 1000$

输出描述

最多获得的短信条数

用例

输入	6 10 20 30 40 60
输出	70
说明	分两次充值最优，1 元、5 元各充一次。总条数 $10 + 60 = 70$

输入	15 10 20 30 40 60 60 70 80 90 150
输出	210
说明	分两次充值最优，10 元 5 元各充一次，总条数 $150 + 60 = 210$

题目解析

本题是 **完全背包问题** 。

如果大家不是很清楚 **完全背包** 的求解，可以看

本题中：

- 客户预算 M 相当于背包的承重，
- 出售价表：
 1. i 元相当于物品的重量，
 2. P_i 短信条数相当于物品的价值

JavaScript 算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const m = parseInt(await readline());
7      const p = (await readline()).split(" ").map(Number);
8
9      const dp = new Array(m + 1).fill(0);
10
```

运行

```

11 |   for (let i = 0; i < p.length; i++) {
12 |       // p[0]条短信花费1元, p[i]条短信花费i+1元
13 |       for (let j = i + 1; j <= m; j++) {
14 |         dp[j] = Math.max(dp[j], dp[j - i - 1] + p[i]);
15 |       }
16 |     }
17 |
18 |     console.log(dp[m]);
19 |   })();

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |
8 |         int m = Integer.parseInt(sc.nextLine());
9 |         int[] p = Arrays.stream(sc.nextLine().split(" ")).mapToInt(Integer::parseInt).toArray();
10 |
11 |         int[] dp = new int[m + 1];
12 |
13 |         for (int i = 0; i < p.length; i++) {
14 |             // p[0]条短信花费1元, p[i]条短信花费i+1元
15 |             for (int j = i + 1; j <= m; j++) {
16 |                 dp[j] = Math.max(dp[j], dp[j - i - 1] + p[i]);
17 |             }
18 |         }
19 |
20 |         System.out.println(dp[m]);
21 |     }

```

Python算法源码

```
1  if __name__ == '__main__':
2      m = int(input())
3      p = list(map(int, input().split()))
4
5      dp = [0 for _ in range(m + 1)]
6
7      for i in range(len(p)):
8          # p[0]条短信花费1元, p[i]条短信花费i+1元
9          for j in range(i + 1, m + 1):
10             dp[j] = max(dp[j], dp[j - i - 1] + p[i])
11
12     print(dp[m])
```

C算法源码

```
1  #include <stdio.h>
2  #include <math.h>
3
4  #define MAX_N 100
5
6  int main() {
7      int m;
8      scanf("%d", &m);
9
10     int p[MAX_N];
11     int n = 0;
12     while (scanf("%d", &p[n++])) {
13         if (getchar() != ' ') break;
14     }
```

```

15
16     int dp[m + 1];
17     for (int i = 0; i < m + 1; i++) dp[i] = 0;
18
19     for (int i = 0; i < n; i++) {
20         // p[0]条短信花费1元, p[i]条短信花费i+1元
21         for (int j = i + 1; j <= m; j++) {
22             dp[j] = (int) fmax(dp[j], dp[j - i - 1] + p[i]);
23         }
24     }
25
26     printf("%d\n", dp[m]);
27
28     return 0;
29 }

```

C++算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int m;
7      cin >> m;
8
9      vector<int> p;
10     int pi;
11     while (cin >> pi) {
12         p.emplace_back(pi);
13     }
14
15     vector<int> dp(m + 1, 0);
16
17     for (int i = 0; i < p.size(); i++) {

```

复制

```
18 |         // p[0]条短信花费1元, p[i]条短信花费i+1元
19 |         for (int j = i + 1; j <= m; j++) {
20 |             dp[j] = max(dp[j], dp[j - i - 1] + p[i]);
21 |         }
22 |     }
23 |
24 |     cout << dp[m] << endl;
25 |
26 |     return 0;
27 | }
```